PA4 — A-Frame VR Scene with Custom Component

AR/VR/XR Applications (6B04103 AI Business) · Write-up E1–E6

Student ID	241177	**Prefix**	`x1177-`
Theme	Winter steppe (7)	**Motion**	orbit (7 mod 3 = 1)
Interaction	point-and-click (1 mod 2 = 1)	**Layout**	corridor (1 mod 2 = 1)

Setup: A-Frame 1.8.0 (THREE r184), Chromium, local HTTPS server, 1280×720. Every scene screenshot shows the ID panel.

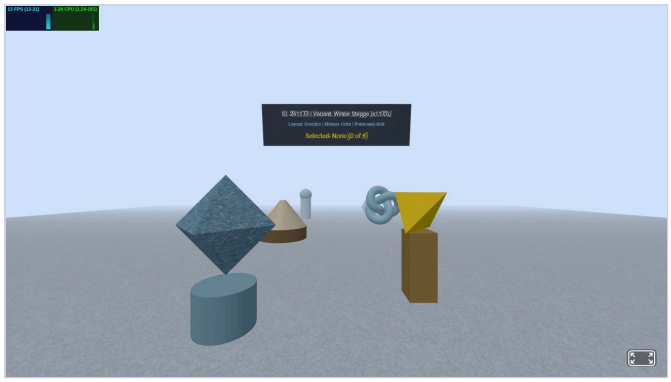


Fig. 1 – Start view: corridor, fog, textured ground and sky, ID panel.

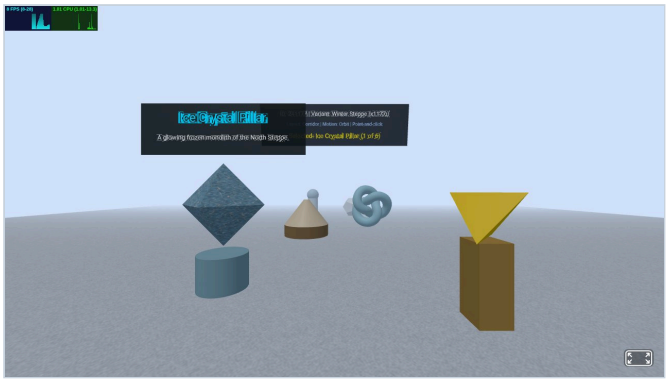


Fig. 2 – Click on Ice Crystal Pillar: info panel, counter “1 of 6”, amplitude 0.4 → 0.8.



Fig. 3 – Selecting Golden Eagle Totem deselects the first (“4 of 6”).

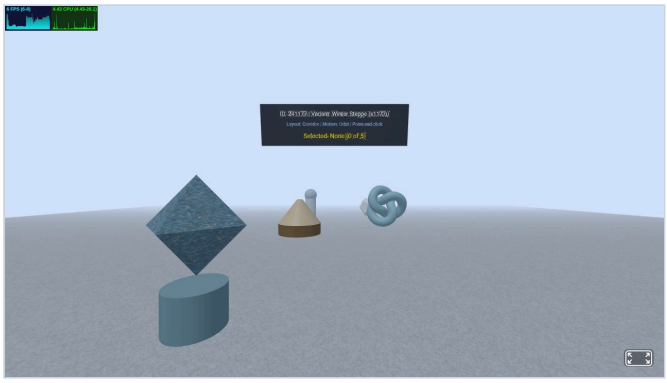


Fig. 4 – After **X**: totem removed from the DOM, counter “0 of 5”.

E1 — The component life cycle

Prediction. `init` then `update` once on load, `play` when the scene starts, one `update` per `setAttribute`, `pause`/`play` on the entity calls, `remove` on `removeAttribute` and again on removing the entity.

Result (`?debug`, entity `#ex1`; run 2 removes `#ex2` with the component still attached):

```
[   390.7 ms] x1177-motion#ex1 init
[   391.3 ms] x1177-motion#ex1 update oldData={} data={"type":"orbit","amplitude":0.4,"period":4,"phase":0}
[   417.6 ms] x1177-motion#ex1 play
--- setAttribute amplitude 0.8
[  4738.4 ms] x1177-motion#ex1 update oldData={...,"amplitude":0.4,...} data={...,"amplitude":0.8,...}
--- el.pause()                    [  5045.4 ms] x1177-motion#ex1 pause
--- el.play()                     [  5346.0 ms] x1177-motion#ex1 play
--- removeAttribute               [  5648.4 ms] pause   [  5648.7 ms] remove
--- remove entity                 (nothing logged)
--- [run 2] remove #ex2           [  6257.0 ms] pause   [  6257.2 ms] remove
```

Explanation. `update` first runs immediately after `init`, with `oldData = {}`, so one code path handles both setup and changes. Afterwards it runs only when parsed data actually differs, and `oldData` holds the previous values (amplitude 0.4). `play` fires once the scene starts, not in `init`. Removing a component always pauses first, then calls `remove`. My prediction was wrong for the last step: after `removeAttribute` nothing is left to log, and run 2 shows that removing an entity also gives pause → remove. Because my motion integrates `angle += ω·dt` and eases amplitude, the 0.4 → 0.8 change produced no jump.

E2 — Checking Ch. 8

(a) Prediction. `setAttribute('ch8-texture','repeat','4 4')` rebuilds `this.data` and loses the texture, material and mesh.

Result (`experiments/ch8-texture.js`):

```
before: texture=Texture 5990e586 material=MeshStandardMaterial 57f30d01 mesh=Mesh 7f03e0e7 repeat={x:2,y:2}
after : texture=Texture 5990e586 material=MeshStandardMaterial 57f30d01 mesh=Mesh 7f03e0e7 repeat={x:4,y:4}
texture.repeat.x on the GPU = 2          schema types: texture/material/mesh = "string"
setAttribute('ch8-texture','texture','oops') → this.data.texture = "oops", mesh still uses old texture
```

Explanation. My prediction was wrong. A-Frame 1.8 patches `data` in place, so the references survived. Still, `data` now says 4×4 while the texture shows 2×2, because the chapter has no `update`. The schema infers type *string* for those three properties, and any `setAttribute` or mixin can overwrite them. The live texture is then orphaned and can never be disposed. The schema is public configuration that A-Frame parses, diffs and serialises. Runtime objects are private state, so they belong on `this`, where only the component can change and dispose them.

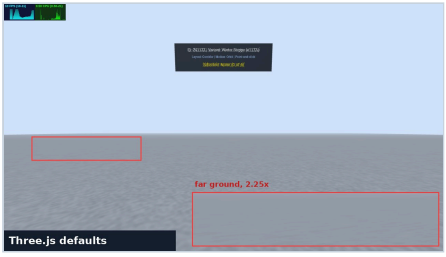
(b) `Object.keys(AFRAME.systems)` has no `assets`. `sceneEl.systems` = camera, geometry, gltf-model, light, material, obb-collider, renderer, shadow, tracked-controls, webxr, **x1177-gallery**. `<a-assets>` is a custom element (`isEntity = false`) that preloads files and delays scene start until `hasLoaded`. `registerSystem` creates one singleton per scene with `init/tick/pause/play`, reachable from every component. So the claim does not hold: `<a-assets>` is an asset manager node, not an ECS system.

E3 — Texture filtering

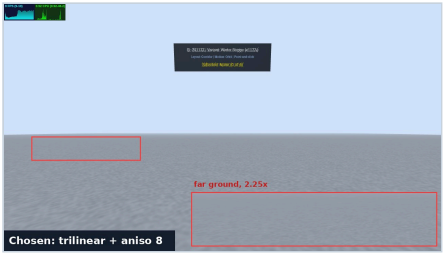
Prediction. The Ch. 8 settings shimmer in the distance, the defaults blur it, and anisotropy keeps it sharp.



Ch. 8: min Nearest, mag LinearMipmapLinear.



Defaults: min LinearMipmapLinear, mag Linear, anisotropy 1.



Chosen: trilinear + anisotropy 8 (max 16).

Result. Camera at 0.4 m, pitch ≈ 1°, ground repeat 10×10. Ch. 8 shows noisy, aliased grain near the horizon. The defaults turn the same area into a smooth grey band. The chosen settings keep the snow detail without the noise. Property check: `'anisotropic' in tex → false` and `'generateMipMaps' in tex → false` (the real names are `anisotropy` and `generateMipmaps`). Mag filter LinearMipmapLinear gives `GL_INVALID_ENUM: Texture filter not recognized`.

Explanation. Minification happens when one pixel covers many texels (far ground), and magnification when one texel covers many pixels (close up). Mipmaps are pre-filtered half-size copies, so minification samples an averaged level instead of skipping texels. NearestFilter without mipmaps causes the shimmer. Magnification has no mipmaps, so a mipmap filter there is invalid. Trilinear filtering blends two mip levels, but at grazing angles it picks a level that is too blurry. Anisotropic filtering takes several samples along the stretched direction. I therefore use LinearMipmapLinear / Linear / anisotropy 8, clamped to `getMaxAnisotropy()`.

E4 — What <a-scene> builds

```
sceneEl.renderer : WebGLRenderer; xr.enabled=false (until Enter VR); outputColorSpace=srgb; pixelRatio=1
sceneEl.object3D : THREE.Scene; 12 children; fog = THREE.Fog(near 4, far 30)
sceneEl.camera   : PerspectiveCamera fov 80, near 0.005, far 10000, owned by #camera inside #rig
webxr system     : referenceSpace=local-floor; sessionInit={"requiredFeatures":["local-
floor"],"optionalFeatures":["bounded-floor"]}
a-scene.js enterVR(): vrManager.setReferenceSpaceType(refspace) → navigator.xr.requestSession('immersive-vr',
xrInit)
                      → renderer.xr.setSession(xrSession) → setFoveation, setWebXRFrameRate, 'enter-vr'
```

Comparison. In PA3 I wrote these steps myself: `navigator.xr.isSessionSupported`, `requestSession('immersive-vr', {optionalFeatures:['local-floor']})`, `renderer.xr.enabled = true`,

`renderer.xr.setSession(session)` and `renderer.setAnimationLoop`. A-Frame does the same thing inside `AScene.enterVR`. It also handles the reference space, foveation, frame rate, the session `end` event, fullscreen on desktop and the Enter VR button. **Raw Three.js** is better for a company product that needs full control of the render loop, custom post-processing or a small bundle inside an existing React or WebGL app. **A-Frame** is better for a fast prototype, a museum or training scene, or a team of web developers, because HTML plus components gives working VR in hours.

E5 — Performance

Prediction. 60 extra exhibits add about 90 draw calls. Instancing should bring that back close to the baseline without reducing triangles.

State	Draw calls	Triangles	fps*
Baseline scene (6 exhibits)	16	12,530	6.0
+60 exhibits (cloned entities)	109	91,812	4.9
60 copies → 10 `THREE.InstancedMesh`	**26**	96,090	4.6

*fps measured in headless Chromium with a software GPU (SwiftShader), averaged over 4 s. The absolute value reflects that renderer. Draw calls and triangles come from `renderer.info` and do not depend on hardware.


Baseline


+60 exhibits (“0 of 66”)


Instanced copies

```
groups by geometry → new THREE.InstancedMesh(geo, mat, n); im.setMatrixAt(i, clone.matrixWorld);
scene.object3D.add(root)
```

Explanation. Every draw call costs CPU time for state changes and driver work, whatever its size. In VR the scene is drawn for two eyes, often as two passes, within 13.9 ms at 72 Hz on a mobile Quest CPU. A missed frame causes judder and nausea, while a laptop simply shows 50 fps. Instancing cut draw calls by 4× (109 → 26) with the same triangles. The trade-off is that the copies share one material and need instance-level raycasting for interaction.

E6 — Comfort and design

The two rows are 3 m apart, so the walkway passes about 1.5 m from each exhibit. That is personal-space distance, close enough to read and far enough not to clip, even when a selected exhibit orbits at 1.0 m. Exhibits are 5 m apart along z, which keeps two in view at once. Text: the ID panel cap height is ≈ 0.14 m at 6.3 m (≈ 1.3°), and the exhibit titles are ≈ 0.17 m seen from 3 m (≈ 3.2°). Both are above the ~1° legibility minimum. Panels are tilted toward the viewer and kept above 2 m. Lasers are on both hands, the raycaster is limited to exhibit meshes with `far: 30`, and the hover scale of 1.15 gives instant feedback. Comfort risk: smooth WASD movement in VR causes vection. I reduced `wasd-controls` acceleration from 65 to 25 and kept the motion slow (periods of 3.5–5 s).

Development log

Bug: hover and click did nothing. I logged `camera.components.raycaster.objects.length` and got **0**. The raycaster only collects the `object3DMap` of matched elements, and a `.exhibit` group has no mesh of its own. Fix: `raycaster="objects: .exhibit > *"`, so events bubble to the exhibit. Commit: `COMMIT_HASH`. Second bug: pressing **M** paused the whole entity and removed its event listeners. Now only `x1177-motion` is paused.